

Test assignment for Rust developer position

Shared on demand. November 2025

This test assignment is developed to evaluate the knowledge, analytical, and practical skills of candidates applying for the Rust developer position.

The test assignment consists of two code tasks. The first task allows you to demonstrate your skills in Rust and software engineering practices. The second task allows you to show your skills in SQL and relational databases. **The second task is optional.** You can skip it and you won't be penalized for it.

Prepare the solutions as a git project. You can share it using GitHub or by sending an archive.

Please, don't share this test assignment publicly.

Task 1

You are building a simple telemetry system which consists of two independent Rust components:

- A sensor node - application which emits telemetry data.
- A telemetry sink - service which receives, processes and saves the telemetry data.

These components must be designed to possibly operate on different machines and be designed to gracefully handle network instability, message overflow, and resource constraints.

Component 1 - Sensor node

The sensor node simulates a remote device generating telemetry data at a configurable rate. Its responsibilities:

- Generate a new sensor value each time frame (e.g. every second).
- Transmit sensor value to the telemetry sink using chosen protocol (gRPC, HTTP, etc).

The sensor value should have the following fields:

- Sensor name.
- Sensor value (integer).
- Timestamp when the value was read.

The node should accept the following configuration (e.g. CLI flags):

- Rate - number of messages per second to send.
- Sensor name - the name of the sensor to use.
- Address of the telemetry sink.

- And others if they are appropriate.

Component 2 - Telemetry sink

The sink receives telemetry from sensor nodes and writes it to a log file. Its responsibilities:

- Receive sensor messages over the network.
- Write incoming messages to a log file. The format is up to you.
- Buffer incoming messages in memory. Messages are flushed to a log file on these events:
 - The buffer is full.
 - A flush timer triggers (e.g. every 100ms).
- Drop incoming messages if the data flow rate exceeds the allowed bandwidth (rate limit in bytes/sec).

The node should accept the following configuration (e.g. CLI flags, env variables, config file):

- Bind address.
- Path to the output log file.
- Buffer size in bytes.
- Buffer flush interval.
- Rate limit - max input flow rate in bytes/sec.

Bonus challenges

We've included a few optional challenges in this task to give you the opportunity to explore areas you're comfortable with and to demonstrate strength in additional areas. They are **not required**, and you won't be penalized for skipping them.

- Support multiple sensors in the Telemetry sink. In this case, the rate limit should be calculated across all sensors.
- Use gRPC for interservice communication.
- Implement graceful shutdown - ensure the buffer is flushed and sockets are closed on SIGINT and SIGTERM.
- Allow configuring TLS (basic) or mTLS (advanced) for interservice communication.
- Encrypt each message before writing it to the log file in the Telemetry sink. Accept encryption key. The choice of cryptographic method is up to you.

What we look for

- Good modular architecture and separation of concerns.
- Clean and readable code.
- Robustness: handling disconnects, dropped messages, malformed data.

- Use of modern Rust idioms.
- Documentation: how to install, configure and run the service.

Task 2 (Optional)

This task includes a core requirement and optional advanced challenges for those seeking an additional challenge.

There are several rooms equipped with sensors that transmit telemetry data (voltage **V** and load **R**). A single room may have multiple sensors simultaneously exporting the same parameter as duplicates. For example, one sensor may transmit **V** and two sensors may transmit **R**.

Each sensor is characterized by a unique name and the type of value it transmits (**V** or **R**). The readings then are stored as floating point values in the database alongside the timestamp of the measurement.

Each room has a unique name and contains several sensors associated with it. Subsequently, the same sensor cannot be located in two different rooms. A single room can contain multiple sensors for different parameters.

Base task

1. Design normalised database scheme. Write SQL to create tables, indexes, foreign keys and constraints if applicable.
2. Create SQL queries to insert the following test data:
 - a. The room **room_A** has 1 sensor for **V** and 2 sensors for **R**.
 - b. The room **room_B** has 2 sensors for **V** and 3 sensors for **R**.
 - c. There is a situation where there is no measurement from one of the sensors if there are multiple sensors in a room (e.g. room had two sensors for **R**, but only one of them sent data at a given point in time).
 - d. There is a situation where there is a measurement for **R** but no measurements for **V** or vice versa.
3. Create SQL query to select data with the following fields:

| room | timestamp | I | V | R |

Where:

$$I = V / R$$

Also:

1. If there are multiple sensors for the same value (e.g. 2 sensors for **V**), the average value across all sensors should be used.
2. The clock is not ideal so two measurements of sensor values can have different timestamps. For example, there could two records like:
 - a. Sensor 1 - 50.1V - 2025-06-30T10:00:00.000001
 - b. Sensor 2 - 50.2V - 2025-06-30T10:00:00.000032

Simple requirements

Each sensor sends one or more measurements per second. The output should group the sensor data by one-second intervals and calculate the average value for each interval.

Advanced requirements (optional)

If the above task seems too simple for you, it can be made more challenging to better demonstrate your skills.

Each sensor sends one or more measurements per second. The main parameter that determines the timestamp for aggregation is **V**. This means the output should include a result for **each aggregated value of V** with a **1-second interval**.

The output should group the sensor data by one-second intervals and calculate the average value for each interval. However, if there are no values for some parameter within the same second, then the **most recent available value from a previous second** should be used.

For example:

timestamp	sensor	V	R
2025-06-30T10:00:01.000001	1	20.1	null
2025-06-30T10:00:01.000002	2	20.2	null
2025-06-30T10:00:00.000001	3	null	5
2025-06-30T10:00:00.000003	4	null	5.1

In this example, we have two **V** values from two devices for the 10:00:01 second. However, there are no **R** values for that same second. In such a case, we should use the closest previous second (i.e., 10:00:00), compute the average of **R** values for that second, and use it when calculating the result **I** for 10:00:01.

What we look for

- Proper normalisation and avoidance of redundancy.
- Clear and appropriate use of types.
- Correct relationships with primary and foreign keys.
- Consistent naming.
- Efficient use of SQL constructs.
- Well-structured and formatted SQL.
- Comments where necessary to explain non-obvious logic.